

43008: Reinforcement Learning

Assignment-3, Part-E Project Report

**Project Title: Mastering the Markets with Deep
Reinforcement**

Learning for Precision Stock Analysis

**Team Name and Project Number: [Foresight Reinforcers,
Project 7]**

**Team Members: [Md Sadman Sakib- 25091570, Karan
Bhutani- 25105521, Giovanni Bianco- 25639648]**

Date: [11 November 2024]

Table of Contents

Abstract	3
1. Introduction and Background	4
2 Overview of the architecture/system	5
3 Results and Evaluation	5
4 Discussion and Conclusions	6
5 References	6
Appendices	7

Abstract

In modern finance, artificial intelligence (AI) and machine learning, especially reinforcement learning (RL), offer significant potential for automating decision-making in stock trading. This project investigates the application of RL techniques—particularly Proximal Policy Optimization (PPO) for stock trading. Our RL agent is designed to dynamically learn and adapt optimal trading strategies in response to various market conditions. This report covers the complete project lifecycle, from data processing to model training, evaluation, and analysis of RL algorithms' performances.

1. Introduction and Background

In this project, we developed a reinforcement learning agent capable of autonomous stock trading. The primary objective was to enable the agent to learn trading strategies that maximize returns while managing risk, an essential aspect in financial markets characterized by volatility and unpredictability. By leveraging RL, the model adapts through interactions within a simulated trading environment, improving over time based on its experience and results.

1.1 Problem Definition

Stock trading requires forecasting price movements and making decisions based on complex data, such as historical prices, technical indicators, and real-time feedback. Traditional models struggle to capture the dynamic nature of markets. RL agents, on the other hand, can learn from large datasets and adapt to changing market conditions by recognizing patterns associated with profitable trades.

1.2 Project Motivation

The motivation for this project is rooted in the potential to build intelligent trading agents that make data-informed decisions independently. By evaluating different RL algorithms, we assess their adaptability and suitability for various market scenarios.

1.3 Application

Beyond academic interest, an efficient RL-based trading model has practical applications for investment firms, financial analysts, and individual investors, offering potential for automated decision-making and portfolio optimization.

1.4 Dataset and Environment

For this study, historical stock data was obtained from Yahoo Finance, focusing on Apple Inc. (AAPL) as a primary case study. The dataset was divided into training, validation, and testing sets, as visualized in Figure 1, providing a structured basis for the agent's training and evaluation phases.

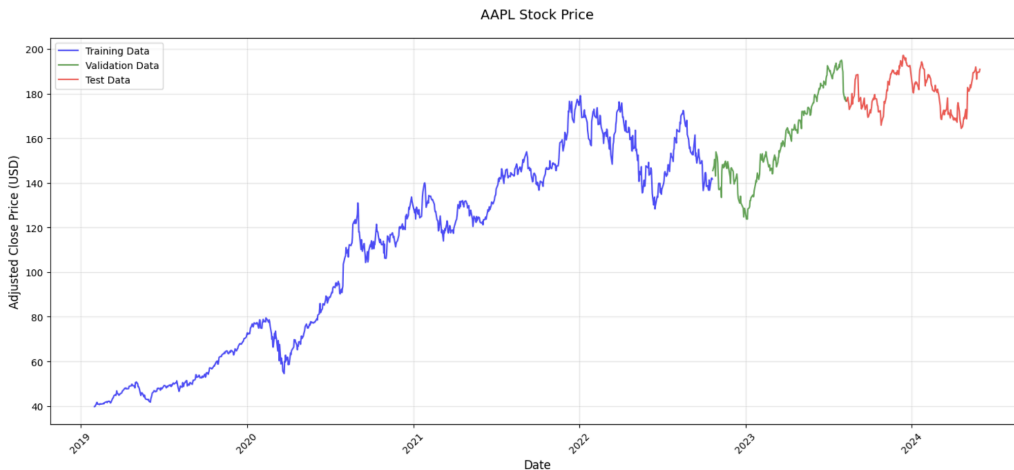


Figure 1. AAPL Stock Price Historical Data Distribution for Model Training

2.1 Data Collection and Preprocessing

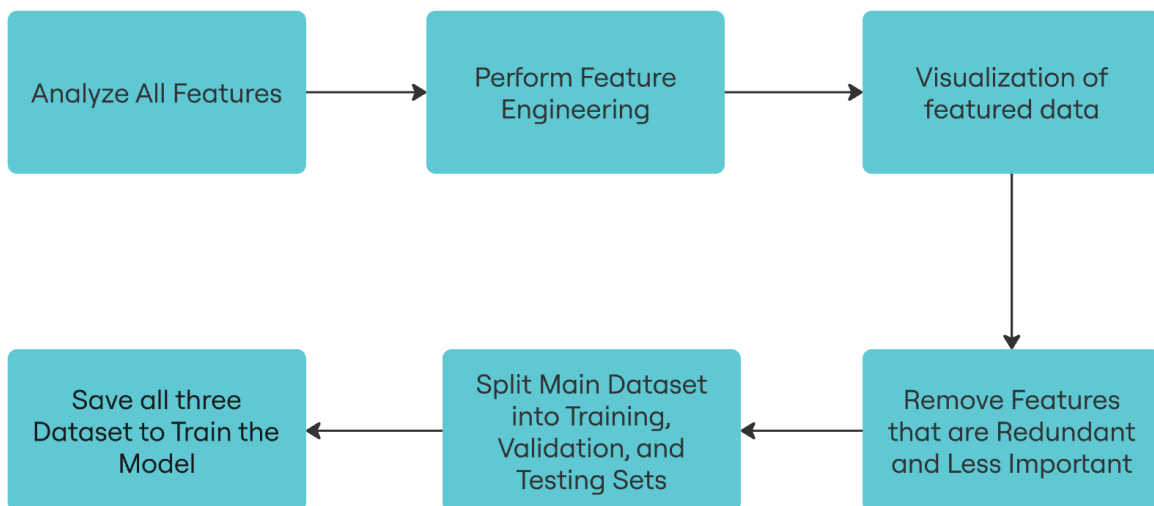
In this project, effective data preparation is paramount for enabling the reinforcement learning agent to make informed trading decisions. The data collection phase involved fetching historical stock data and constructing a robust feature set through careful preprocessing, encompassing technical indicators that encapsulate essential aspects of market behavior.

2.1a. Fetching Data

Historical data was obtained from Yahoo Finance, focusing on a range of high-liquidity stocks within the S&P 500 index, with Apple Inc. (AAPL) serving as a case study. Data was collected from 2019 to mid-2024, a period chosen for its inclusion of both bullish and bearish market cycles, thus providing the agent with exposure to diverse trading conditions. The dataset was systematically partitioned into training, validation, and test sets:

- **Training:** 70% of the data, used to teach the agent and fine-tune model parameters.
- **Validation:** 15% of the data, employed to validate the model's generalization during training.
- **Testing:** 15% of the data, used exclusively for evaluating final model performance.

This segmentation ensures that the agent encounters realistic market conditions in each phase, simulating real-world trading scenarios.



2.2 System Architecture Design

The reinforcement learning (RL) environment is a critical component in training the agent to interact with the stock market. For this project, we implemented a custom stock trading environment using OpenAI Gym's interface, tailored specifically to simulate the intricacies of trading dynamics. This environment, built around the unique features of stock trading, serves as a sandbox where the agent can learn from market feedback by taking actions, observing outcomes, and adjusting strategies accordingly.

2.2a. Action and Observation Space

The environment defines both **action** and **observation** spaces to represent the agent's possible moves and the information it perceives at each timestep:

- **Action Space:** The action space is continuous, bounded between -1 and 1, where:
 - **-1** represents selling all shares, signaling the agent's decision to exit its position.
 - **1** represents buying shares with the maximum available balance, signaling the agent's decision to go fully long.
 - **0** represents a hold action, indicating no change in the current position.
- This continuous action space enables the agent to make granular trading decisions by controlling the extent of buying or selling at each step. Such flexibility allows the agent to react to subtle changes in market conditions, which is crucial for environments with continuous price movements like stock trading.
- **Observation Space:** The observation space is constructed to capture both market and account information:
 - Market features include essential stock price indicators (like **Adj Close**) and derived technical indicators (**EMA_5_20_Ratio**, **RSI_14**, **MACD**, etc.), providing the agent with a comprehensive view of the current market state.
 - Account features include **balance**, **net worth**, **shares held**, **cost basis**, and **total shares sold**, providing a holistic understanding of the agent's financial status.
- The combined observation space represents the agent's knowledge of both external market conditions and its internal financial state, simulating the decision-making context a trader would use. This combination is essential for the agent to learn how market actions affect its portfolio over time.

2.2b Reward Function

The reward function is central to reinforcing profitable behaviours in the agent's policy. Here, we defined a reward function based on changes in **net worth** to align with the goal of maximizing returns:

- **Net Worth-Based Reward:** The agent's reward at each timestep is calculated as the change in net worth (the combined value of balance and held assets) compared to the previous step. This reward structure incentivizes the agent to pursue actions that yield an overall increase in net worth.

- **Risk and Reward Balance:** By defining rewards in terms of net worth rather than immediate profit, the agent is encouraged to adopt a long-term perspective, optimizing strategies to maximize cumulative wealth rather than short-term gains. This approach aligns with the goals of risk-adjusted returns and portfolio stability, critical in volatile markets.

2.2c Trading Constraints and Costs

To simulate realistic trading conditions, the environment includes transaction costs and trading constraints that impact the agent's actions:

- **Transaction Costs:** Each buy or sell action incurs a small percentage cost (e.g., 0.1%). These costs are deducted from the agent's balance, effectively penalizing high-frequency trading and encouraging the agent to make well-considered decisions. This constraint reflects real-world trading scenarios where transaction fees affect profitability.
- **Account Constraints:** The environment tracks essential account variables:
 - **Balance:** The liquid cash available to the agent for buying assets.
 - **Shares Held:** The number of shares the agent currently holds.
 - **Cost Basis:** The average purchase price of held shares, used to calculate gains or losses.
 - **Total Shares Sold and Total Sales Value:** Metrics that track the cumulative impact of sales over time, providing insight into realized gains.
- These constraints ensure that the agent's actions are bound by realistic account mechanics, preventing it from executing financially infeasible strategies.

2.2d State Representation and Next Observation

Each step in the environment begins with the agent perceiving a new **state** or **observation**, which includes both market and account data. The environment's `_next_observation` method compiles this state by:

- Extracting the market data for the current timestep, excluding irrelevant columns (e.g., `Date`).
- Merging this data with account variables to provide a holistic view of both market conditions and the agent's financial status.

The combined state space enables the agent to understand the impact of its trading actions on portfolio value, a critical feedback mechanism for learning.

2.2e Step Function and Trading Logic

The `step` function defines the primary logic of the environment, controlling the agent's interactions with the market:

- **Executing Actions:** Based on the agent's action, the environment calculates how many shares to buy or sell, constrained by the current balance and stock price. Buying actions reduce balance and increase shares held, while selling actions increase balance and reduce shares held.

- **Updating Financial Metrics:** After each action, the environment updates the agent's net worth and balance. These metrics allow the agent to track the outcomes of its trading decisions over time.
- **Termination Condition:** The episode terminates when the agent reaches the end of the dataset, providing a finite horizon that aligns with typical backtesting scenarios in finance.

The **step** function also calculates the reward for the agent based on changes in net worth, feeding this feedback back into the agent's policy.

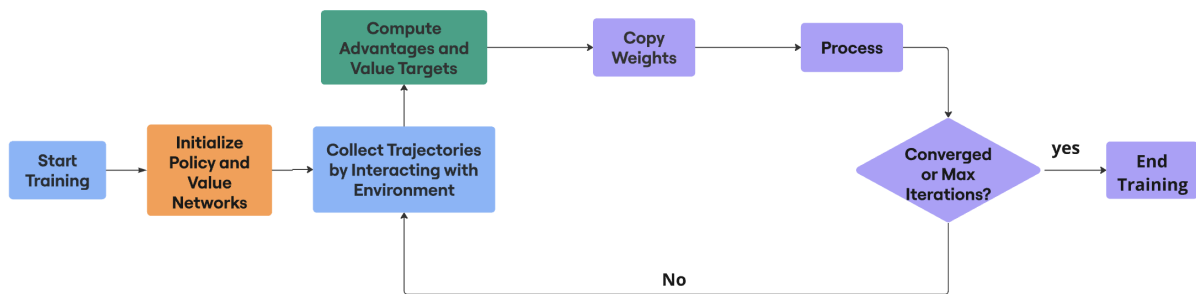
2.2f Custom Visualizations and Metrics

To facilitate performance monitoring and analysis, the environment includes custom visualizations and metrics:

- **Net Worth Tracking:** The environment captures net worth progression over time, visualizing the agent's performance across different market conditions. This metric serves as a proxy for cumulative profitability, allowing the comparison of different models' strategies.
- **Policy Gradient and Loss Monitoring:** During training, metrics like policy gradient loss and model stability are tracked, especially for algorithms like PPO. Monitoring these metrics helps ensure that the agent's policy is stable and converging, crucial for long-term profitability.

The environment also supports testing across multiple agents, enabling side-by-side comparisons of strategies. This setup is valuable for validating the performance of each model and identifying the strengths and weaknesses of individual algorithms.

Figure 2.PPO Training Architecture



In this experiment we started with TD3, and we finally chose PPO. PPO is a policy gradient algorithm that uses a clipped objective function, ensuring stable training in continuous action spaces like stock trading.

- **Network Architecture:** MLP policy with three hidden layers to capture complex market dynamics.
- **Advantages:** Stable and conservative in decision-making, suited for lower-risk trading.
- **Limitations:** Lacks agility in volatile markets.

2.3 GUI/Environment Design

The graphical user interface (GUI) for this autonomous stock trading simulation was built using Streamlit, allowing for an interactive and visually engaging experience. This GUI facilitates the user's ability to configure a simulated trading environment, select stocks, observe trading outcomes in real time, and monitor key metrics. Below is an overview of the design, components, and functionality.

Main Layout and Style

- The page is set up with a **wide layout** for an immersive view of the simulation.
- Custom CSS is applied to enhance visual appeal, featuring a **light background** (`#f4f4f9`) with contrasting colors for text (`#333`) and highlights (`#E74C3C`). This color scheme enhances readability and gives a clean, professional look to the interface.

Sidebar Configuration

- The **sidebar** provides options for setting up the simulation:
 - **Stock Selection:** Users can select multiple stocks from popular choices such as AAPL, GOOGL, MSFT, TSLA, and others. The selected stocks form the portfolio that the trading agent will manage during the simulation.
 - **Date Range Selection:** Users can specify a start and end date for the simulation, allowing the agent to operate over historical data from Yahoo Finance.
 - **Run Simulation Button:** Initiates the simulation based on the selected configuration.

This setup allows users to customize their simulation environment easily, encouraging experimentation with different portfolios and timeframes.

Key Components in the Simulation Environment

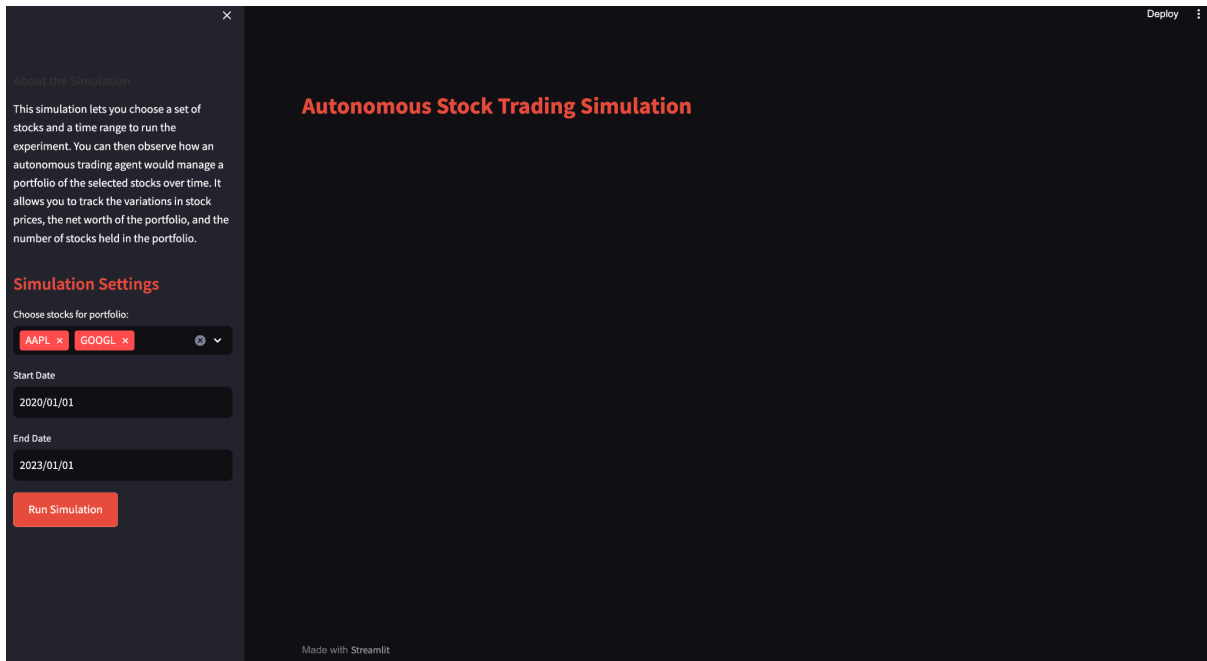
1. **Data Fetching and Initialization:**
 - The `fetch_stock_data` function downloads historical price data for the selected stocks over the specified date range, utilizing the `yfinance` library.
 - The `StockTradingEnv` class, implemented using the Gym framework, manages the trading environment. It initializes the account balance, holds stock data, and tracks metrics like net worth, balance, and shares held for each ticker.
2. **Observation and Action Space:**
 - The observation space includes each stock's price data, along with portfolio-related information such as balance, shares held, net worth, maximum net worth, and the current step in the simulation.

- The action space is a continuous value between -1 and 1 for each stock, representing buy, hold, and sell actions based on the agent's trading strategy.
3. **Real-Time Simulation and Plotting:**
- The **simulate** function iterates through each trading step, updating net worth, shares held, and stock prices based on the agent's actions.
 - **Matplotlib** is used to plot results dynamically:
 - **Net Worth Over Time:** Shows the evolution of portfolio net worth, giving users insight into the agent's profitability and performance consistency.
 - **Number of Shares Held Over Time:** Displays the number of shares held for each stock in the portfolio, helping users visualize the agent's trading decisions.
 - **Stock Prices Over Time:** Tracks price movements of each stock, offering a clear comparison between stock performance and the agent's trading actions.
4. **Real-Time Metrics and Feedback:**
- Real-time metrics, such as **current net worth** and **shares held per stock**, are updated with each simulation step. These metrics are displayed using Streamlit's markdown functionality with customized HTML styling to highlight financial changes.
 - A **progress bar** reflects the simulation's progress, creating an engaging experience by providing users with a sense of time and completion.
5. **Environment Rendering and User Feedback:**
- The agent's balance, shares held, net worth, and profit are printed to the console each step, allowing users to monitor the state of the portfolio at a granular level.
 - The environment's **render** function further provides a summary of portfolio changes, aiding in analysis of the agent's trading logic.

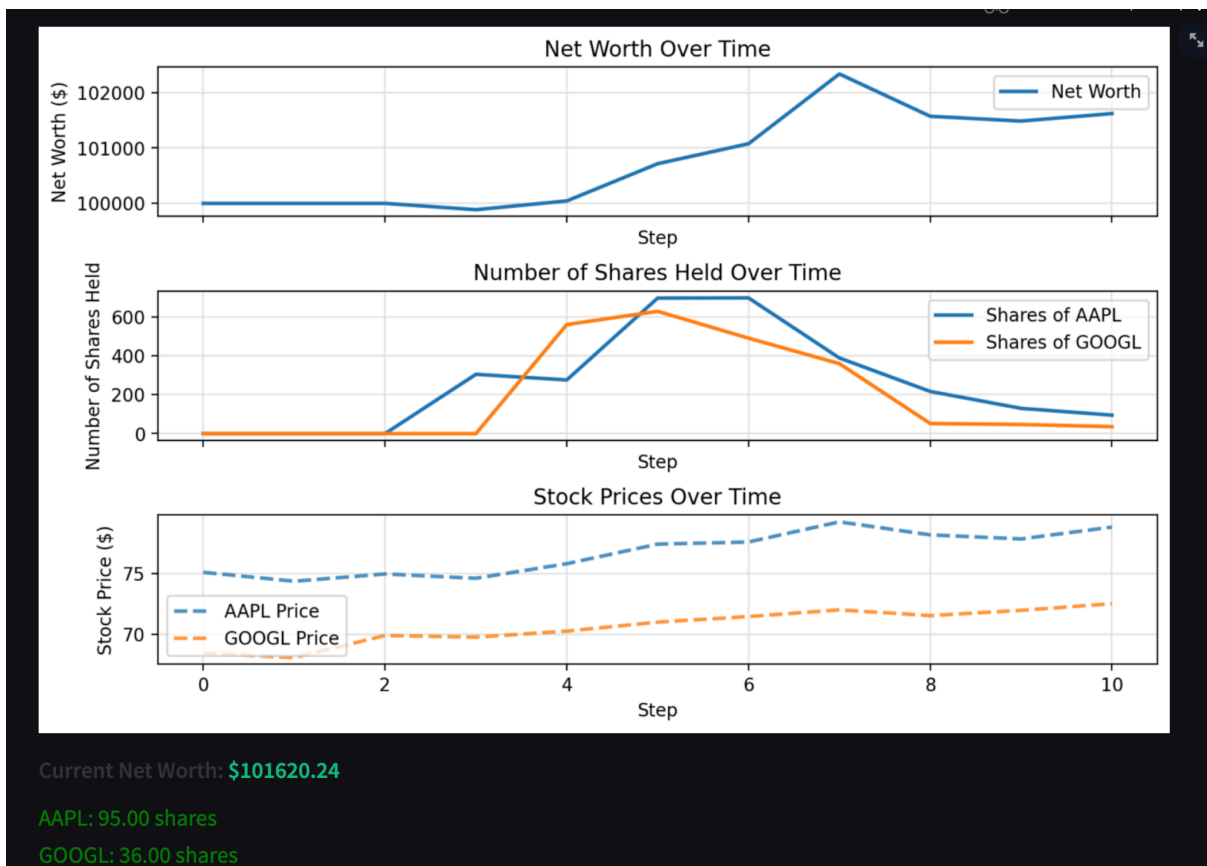
Screenshots and Visuals

The GUI includes several interactive and visual components that enhance the user experience:

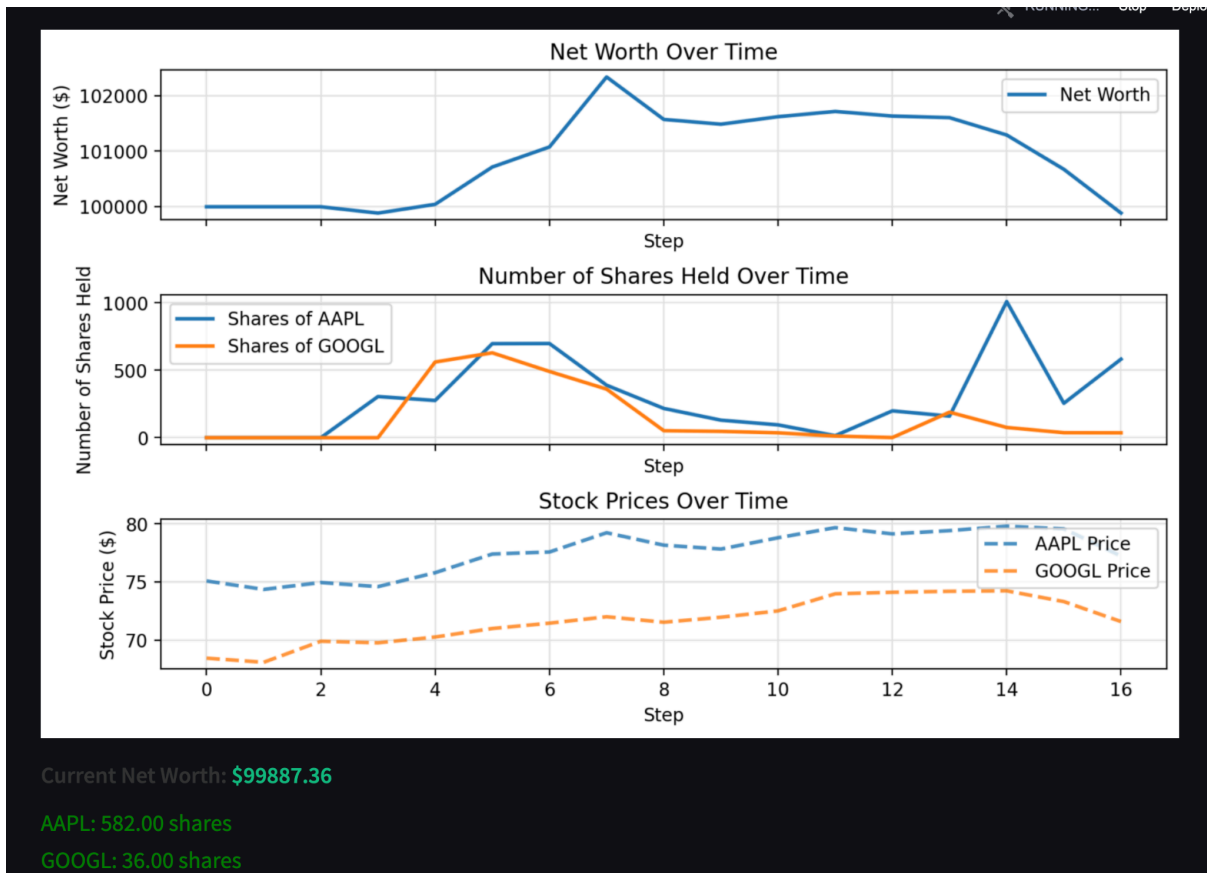
- **Screenshot of Sidebar:** Showcases the stock selection and date input fields, emphasizing the customization options available to users.



- **Net Worth Over Time Plot:** Displays portfolio value evolution, giving users a clear view of the agent's financial performance.



- **Shares Held and Stock Prices Over Time Plots:** These plots illustrate the agent's trading activity and how stock prices fluctuate during the simulation, providing a visual narrative of market dynamics.



3 Results and Evaluation

Our experiments evaluated the effectiveness of the PPO-based trading agent using both training metrics and out-of-sample test data. The results demonstrate the model's capability to maintain stable performance while managing risk in dynamic market conditions.

3.1 Experimental Settings

The PPO agent was trained with the following key parameters:

- Learning rate: 3×10^{-4}
- Total timesteps: 5 million
- Evaluation frequency: 10,000 steps
- Early stopping configuration:
 - Maximum evaluations without improvement: 10
 - Minimum evaluations required: 10

The training process was monitored using Tensor Board logging, and models were automatically saved when improvements were detected. Below is the entropy loss plot showing the training progression.

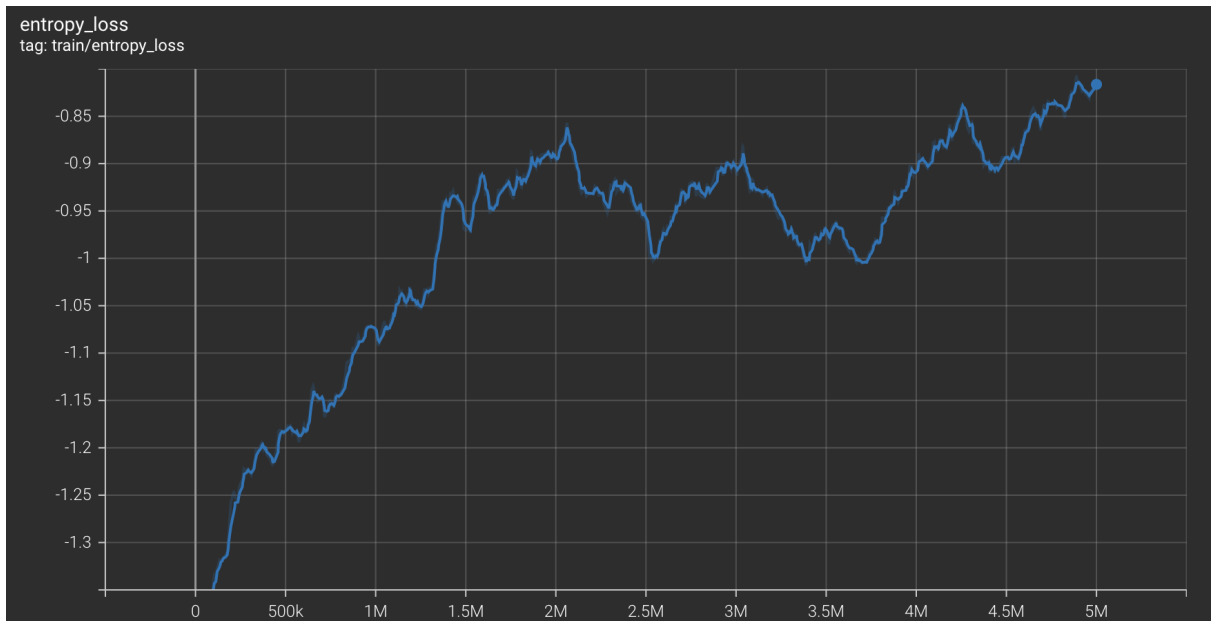


Figure 2.

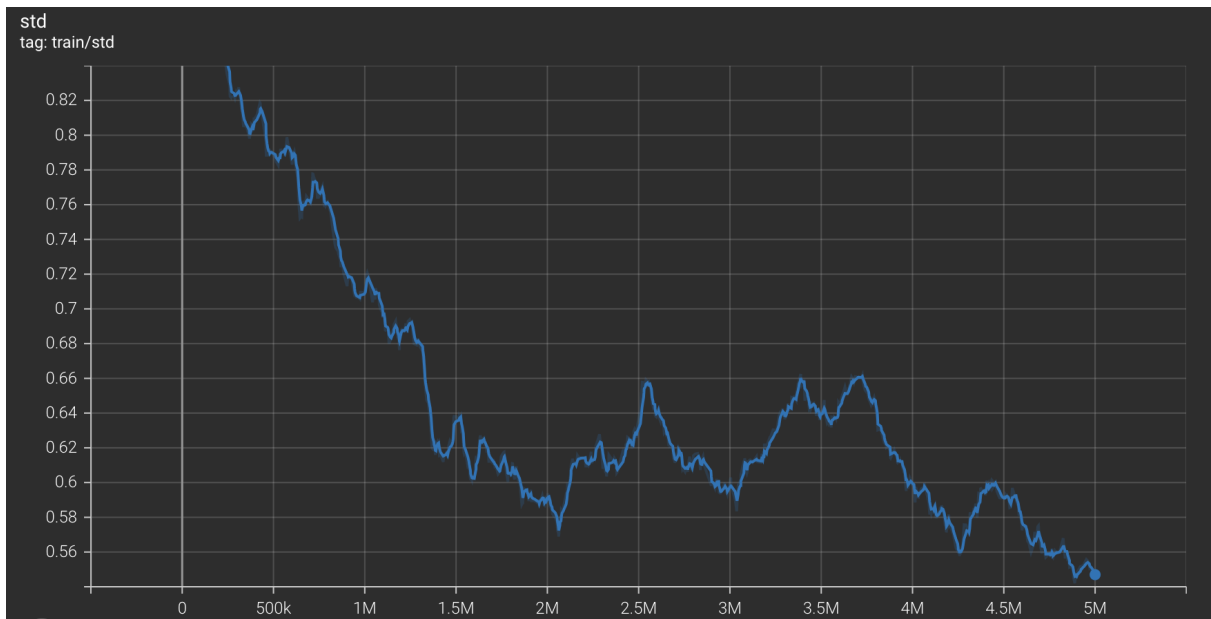


Figure 3. Standard Deviation (std)Plot

Standard Deviation (std): The plot shows a clear decrease in standard deviation from approximately 0.82 to 0.55 over the 5 million timesteps. This declining trend indicates:

- Initial high exploration phase (0-1M steps) with high variability in actions
- Gradual stabilization of the policy (1.5M-3M steps)
- Final convergence phase (3M-5M steps) with reduced action variability, suggesting the agent has learned a more deterministic policy

3.2 Pre-Processing

Several preprocessing steps were performed based on the loaded dataset to capture the real indicators. Correlated features that are similar were not used. Therefore, features that showed high correlation among themselves were dropped.

3.2a Feature Engineering

The selected features and technical indicators capture essential aspects of stock price dynamics, including momentum, trend, and volatility. Each feature was chosen based on its correlation with the target variable "Returns," and its potential to enhance the agent's understanding of market behavior. Key features include:

- **EMA_5_20_Ratio**: The ratio of short-term to medium-term Exponential Moving Averages (EMA) provides insights into short-term momentum against the broader trend, helping the agent detect trend reversals.
- **Price_to_MA20**: The ratio of the current price to the 20-day Simple Moving Average (SMA) acts as a momentum indicator, with higher values signaling bullish momentum.
- **RSI_14 (Relative Strength Index)**: A momentum oscillator that detects overbought or oversold conditions, aiding the agent in anticipating potential reversals.
- **MACD (Moving Average Convergence Divergence) and MACD_Hist**: MACD and its histogram capture momentum shifts by measuring the difference between two moving averages, providing insights into trend direction.
- **Volatility_10**: The 10-day volatility captures price variability, giving the agent an indication of potential risks or market instability.
- **Volume_scaled and Volume_ROC**: Scaled volume and rate of change in volume signal trading intensity, useful for identifying high-activity periods that often correspond to significant price movements.

Each of these features was selected after analyzing its distribution, skewness, and kurtosis, and its correlation with returns. This careful selection ensures the agent can assess both trend strength and market volatility, enabling a nuanced understanding of potential trading opportunities.

3.2.b Data Normalization and Handling Correlations

To mitigate the risks of multicollinearity and ensure consistent scale across features, several preprocessing techniques were applied:

- **Scaling**: Features like volume and adjusted closing price were standardized, improving model stability and convergence during training.
- **Correlation Analysis**: Highly correlated features (e.g., **Adj Close**, **Close**, **High**, **Low**, **Open**) were carefully managed to prevent redundancy. This analysis helps retain only the most informative features, balancing model complexity and interpretability.
- **Distribution Analysis**: Features were examined for skewness and kurtosis to assess the need for transformations. For instance, volume-related features showed high positive skewness, indicating infrequent but significant spikes in trading activity.

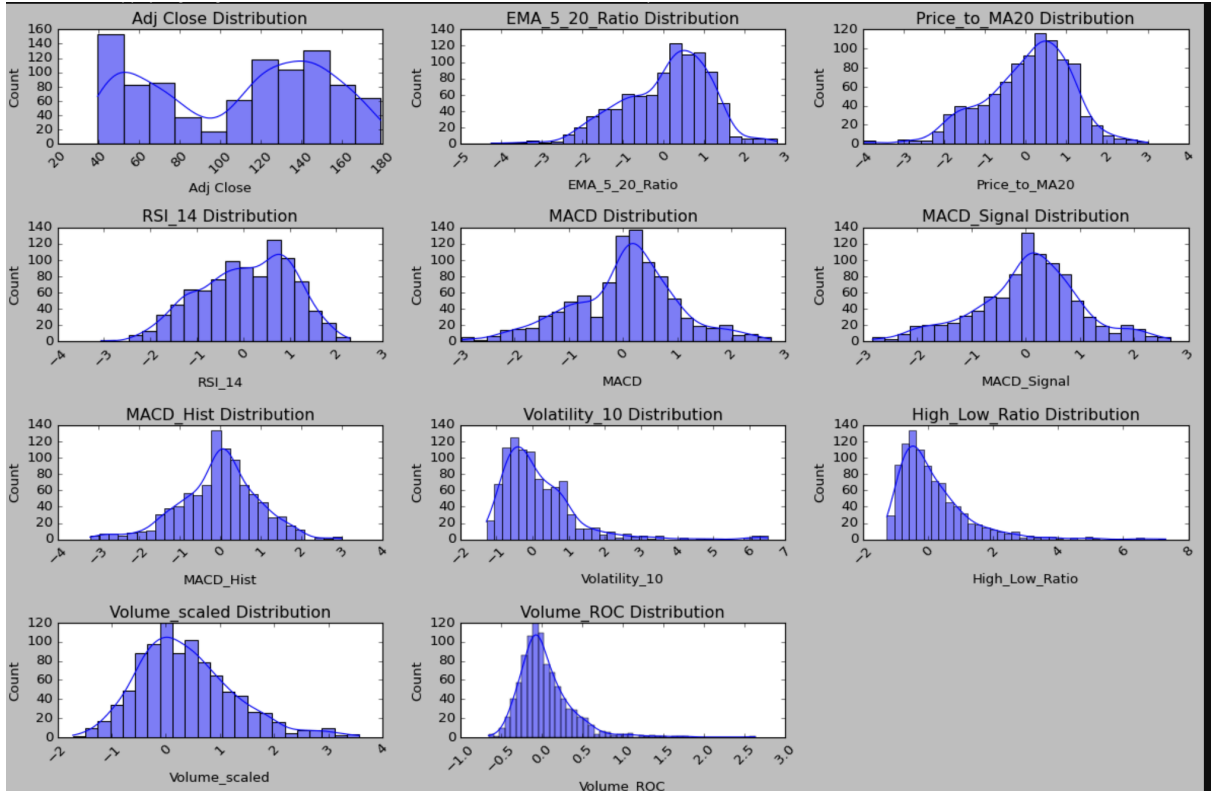


Figure 4:

3.3 Experimental Results

Training Performance

During the training phase, the PPO agent was trained over 5 million timesteps using the AAPL training dataset. The evolution of the agent's mean reward during training is illustrated in Figure 3.

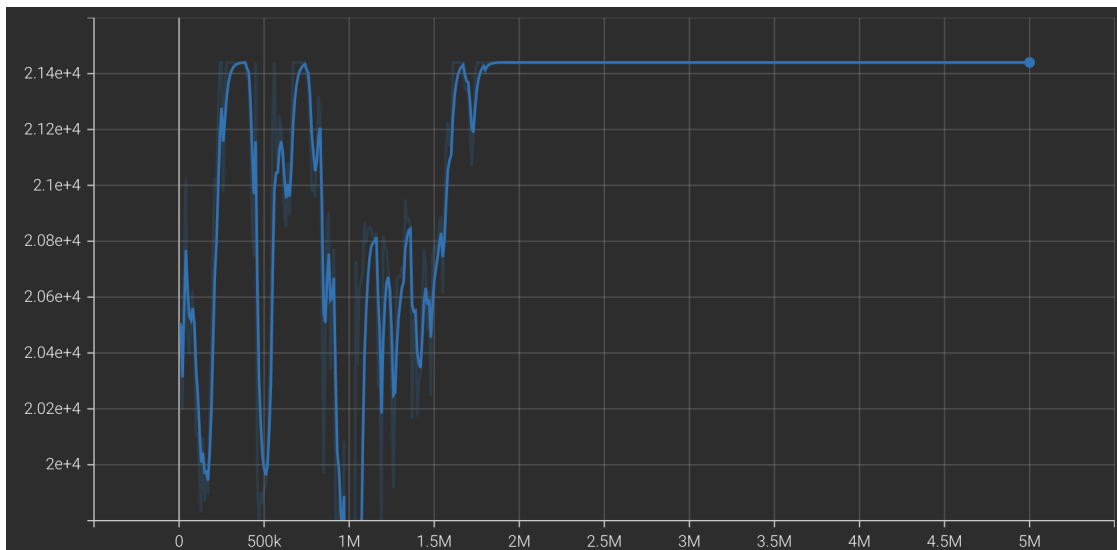


Figure 5: PPO Training Performance: Mean Reward Evolution Across 5M Steps

Figure 3 shows that the mean reward experienced significant fluctuations during the initial phase of training, indicating the agent's exploration of the action space and adaptation to the trading environment. As training progresses, the mean reward stabilizes and gradually increases, suggesting that the agent is learning a policy that yields higher returns.

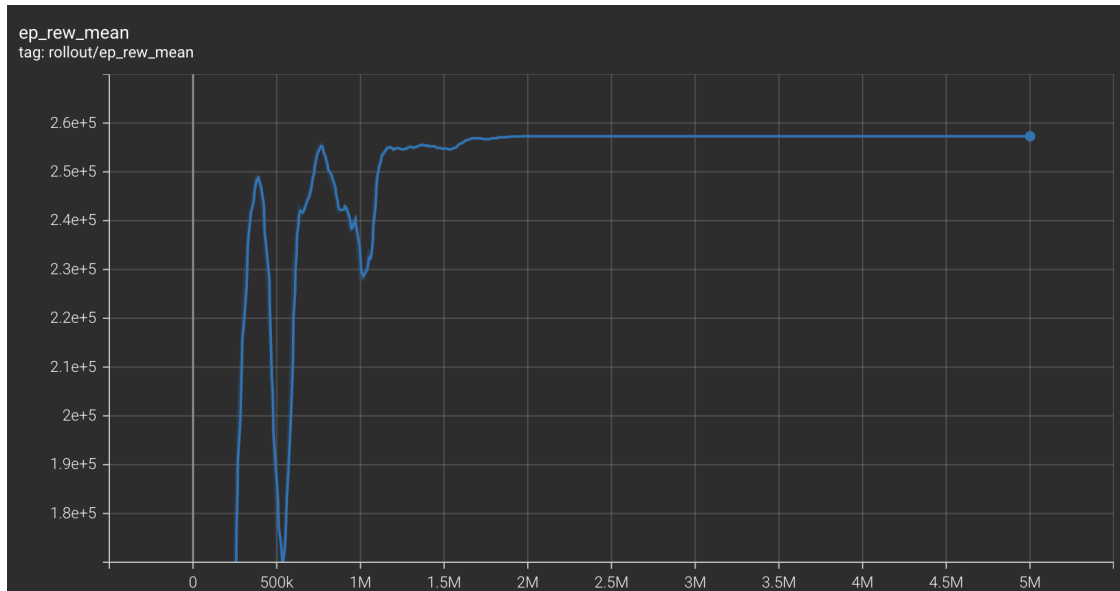


Figure 6: Mean Episode Reward Distribution on Validation Dataset (5M Steps)

To evaluate the agent's performance on unseen data during training, we monitored the mean episode reward on the validation dataset. This is depicted in **Figure 4**.

Figure 4 demonstrates an upward trend in the mean episode reward over time, indicating improved generalization and performance on the validation set. The narrowing distribution reflects the agent's increasing consistency in achieving higher rewards.

Testing Performance

After training, the agent's performance was evaluated on the test dataset, which was not used during training or validation. The key trading performance metrics are summarized in **Table 1**.

Table 1: Summary of Trading Performance Metrics

Metric	Value
Total Return	7.34%
Max Drawdown	-16.60%
Sharpe Ratio	0.54
Sortino Ratio	0.86
Volatility	20.81%
Win Rate	52.5%

The agent achieved a total return of 7.34% over the testing period, demonstrating its ability to generate profits. The maximum drawdown of -16.60% indicates the largest decline from a peak, reflecting periods of adverse market conditions and the agent's exposure to risk.

The Sharpe ratio of 0.54 suggests moderate risk-adjusted returns, while the Sortino ratio of 0.86 indicates better performance when considering only downside volatility. The annualized volatility of 20.81% highlights the variability in the agent's returns. A win rate of 52.5% shows that the agent made profitable trades slightly more than half the time.

Analysis of Results

The experimental results indicate that the PPO-based agent is capable of learning and executing trading strategies that yield positive returns. The agent successfully increased the initial portfolio value, as evidenced by the portfolio value and cumulative returns plots.

However, the agent experienced significant drawdowns, with a maximum drawdown of -16.60%. This suggests that while the agent can capitalize on profitable opportunities, it is also susceptible to substantial losses during unfavorable market conditions.

The moderate Sharpe and Sortino ratios imply that the returns achieved are accompanied by considerable volatility. The agent's strategy results in frequent small gains and losses, as shown in the daily returns distribution, which may limit the overall risk-adjusted performance.

The win rate close to 50% indicates that the agent's decision-making leads to a nearly even split between profitable and unprofitable trades. Improving the precision of trade entries and exits could enhance profitability.

Overall, while the agent demonstrates the potential of reinforcement learning in stock trading, there is room for improvement in terms of risk management and consistency of returns.

3.4 Limitations

The performance metrics and visualizations indicate several areas where the Proximal Policy Optimization (PPO) trading agent can be further refined. Key limitations identified are:

1. Moderate Total Return with High Volatility:

- The agent achieved a total return of 7.34%, which, while positive, indicates that the strategy's profitability could be improved. The “Portfolio Value Over Time” plot shows significant fluctuations, reflecting the agent’s exposure to market volatility.

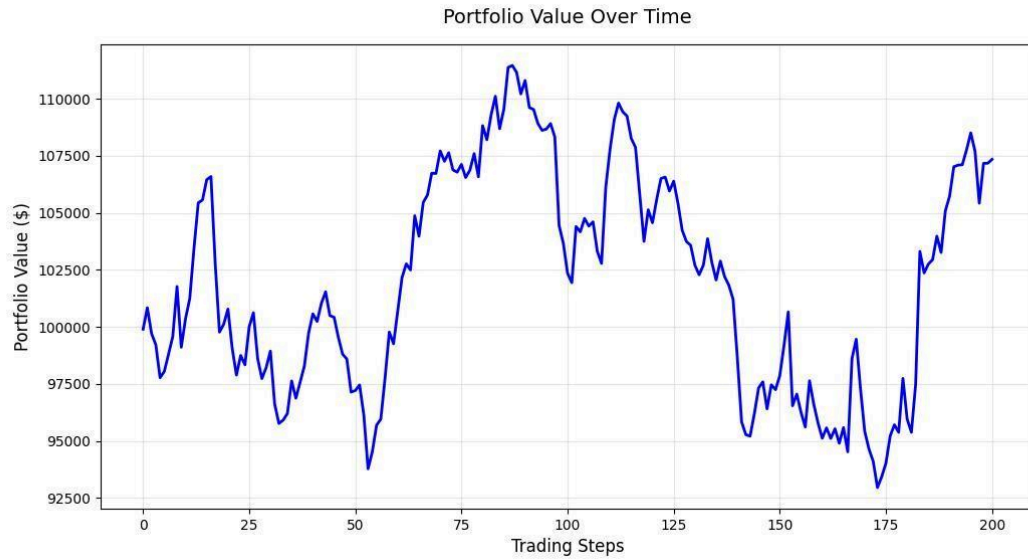


Figure 7: Portfolio over time

- The volatility of 20.81% highlights the inherent risk in the agent's decisions, as demonstrated by the portfolio's drawdown periods. The "Drawdown Over Time" chart shows multiple sharp declines, with a maximum drawdown of -16.6%, indicating the agent's vulnerability during adverse market conditions.

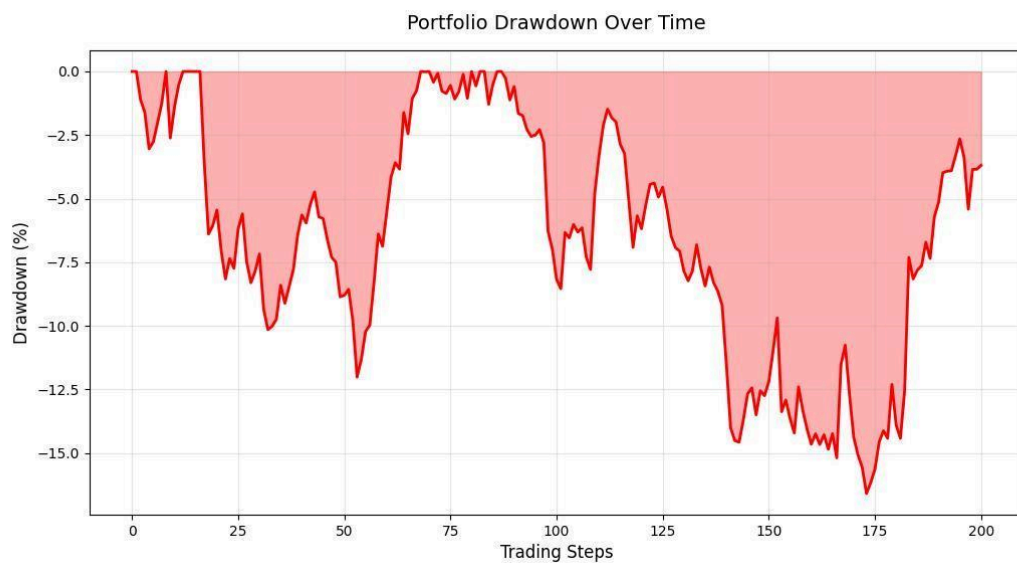


Figure 8: Drawdown over time

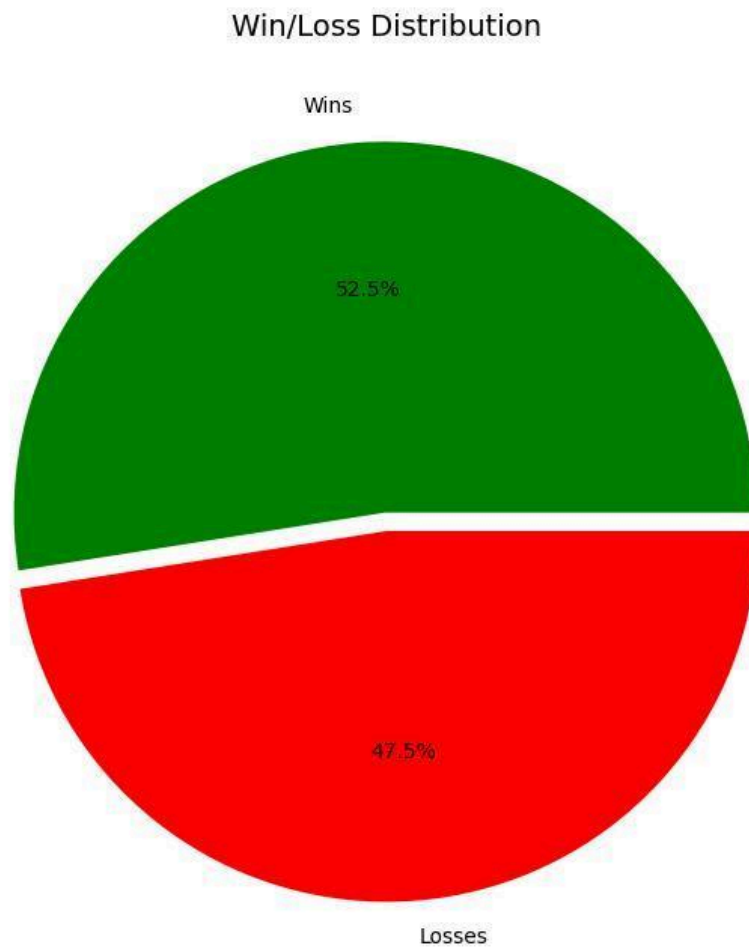
2. Limited Responsiveness to Changing Market Trends:



Figure 9: Cumulative returns over time

- The “Cumulative Returns Over Time” chart illustrates periods of negative returns, suggesting that the PPO model may struggle to adapt swiftly to sudden market shifts. This may stem from the PPO’s conservative approach, which prioritizes stable policy updates over rapid adjustments.
- While this approach helps in maintaining steady policy behavior, it can lead to missed opportunities during volatile market phases where quick adaptation is necessary.

3. Risk-Adjusted Performance Could Be Improved:



- The Sharpe ratio of 0.54 and Sortino ratio of 0.86 are relatively modest, indicating that the agent's returns do not adequately compensate for the risks taken. The "Daily Returns Distribution" shows a narrow range around zero, with a relatively even spread of winning and losing days as seen in the "Win/Loss Distribution" pie chart (52.5% wins vs. 47.5% losses).
- These metrics suggest that the agent's strategy lacks the robustness required for consistent high returns, especially when considering the variability in its daily returns.

4. Simplified Transaction Costs and Execution Assumptions:

- The model assumes a transaction cost of 0.1% for each trade, which does not fully capture the real-world trading costs like slippage and liquidity constraints. This simplification may lead to an overestimation of profitability in scenarios where high-frequency trading incurs greater costs.
- The reliance on historical prices as the basis for action execution further limits the model's realism, as real-time conditions in live markets can vary.

4 Discussion and Conclusions

This project investigated the application of Proximal Policy Optimization (PPO) to automated stock trading, with an emphasis on maximizing returns in a simulated trading environment. The visualizations provide valuable insights into the agent's strengths, limitations, and areas for future improvement.

Summary of Findings

1. PPO's Conservative Yet Stable Approach:

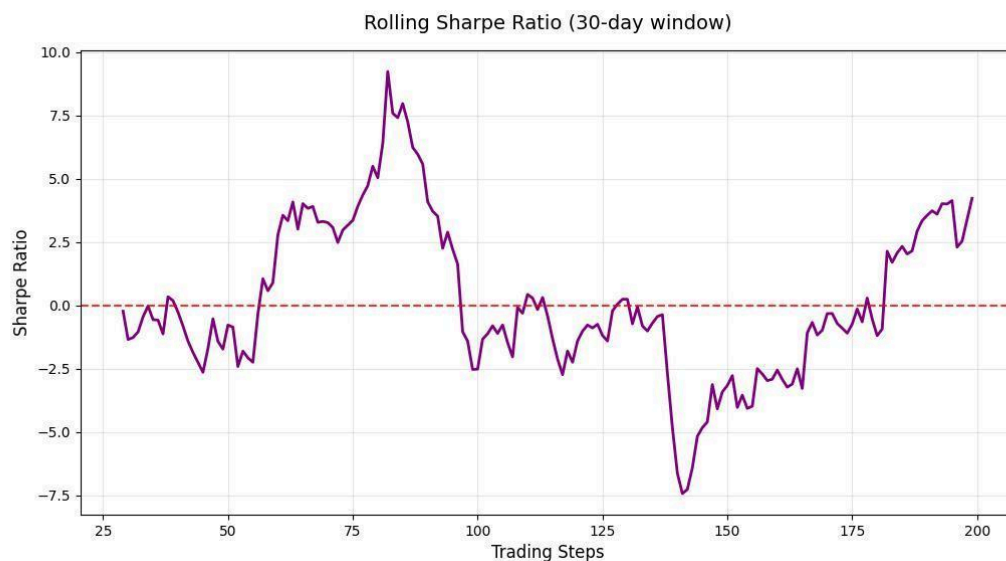


Figure 10: Rolling Sharpe Ratio

- The PPO agent exhibited stable policy learning, as evidenced by the “Entropy Loss” plot, which shows a steady reduction in exploration, indicating convergence toward a consistent policy. This stability aligns with PPO’s design, which limits drastic policy changes and results in a more conservative decision-making process.
- The “Rolling Sharpe Ratio” plot demonstrates that the model's risk-adjusted return fluctuates around zero initially, eventually stabilizing in positive territory, albeit at a modest level. This suggests that PPO’s conservative nature helped maintain a stable return profile without large spikes in risk exposure.

2. Reward Structure and Strategy Balance:

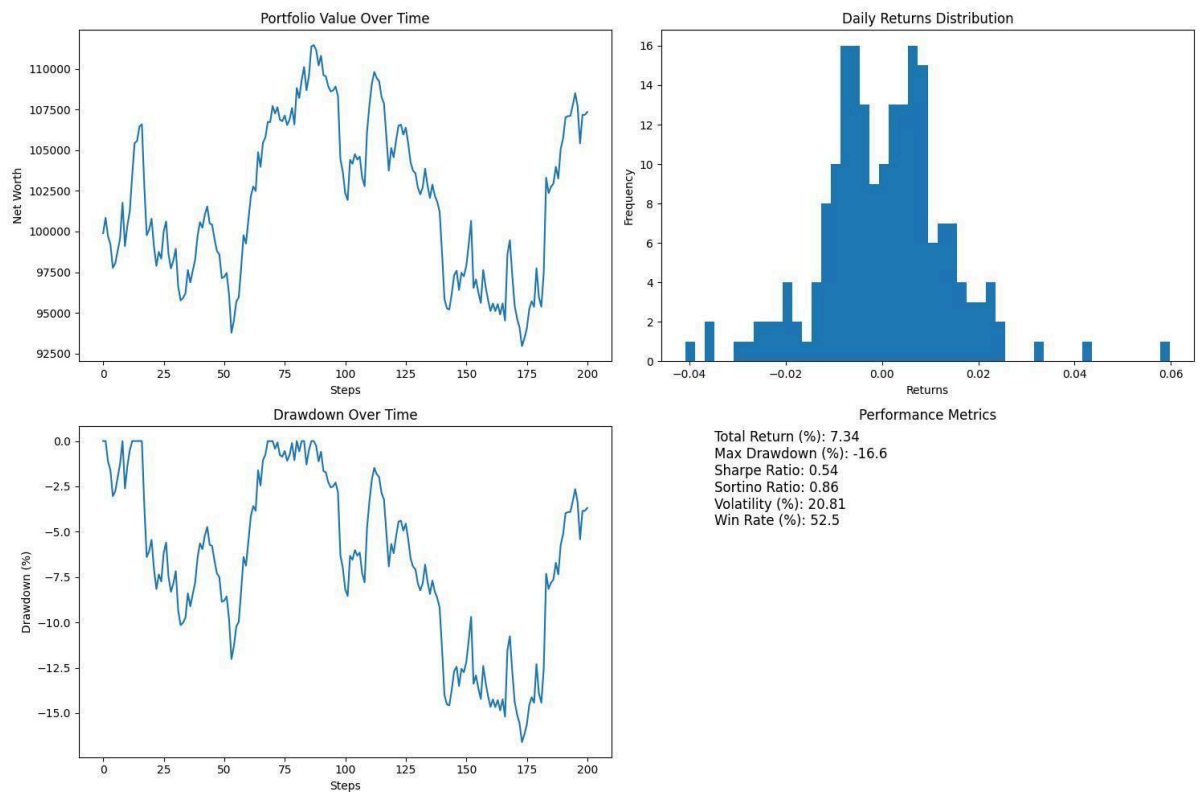


Figure 11:

- The reward structure, primarily based on net worth change, encouraged the agent to focus on cumulative wealth growth. The “Mean Reward” plot shows periods of high variability before stabilizing around 2.1×10^4 , indicating that the agent's strategy converges but may be limited in its ability to capture dynamic market patterns.
- The “Mean Episode Reward” plot reinforces this, showing a significant rise during early training, followed by stabilization. This trend reflects the agent's learning curve, suggesting that while it quickly adapted to the environment, further improvements in reward scaling could incentivize risk-adjusted strategies more effectively.

3. Detailed Metric Insights:

- The “Portfolio Drawdown Over Time” plot emphasizes the model's vulnerability during drawdown phases, which negatively impacts the agent's overall profitability. Additionally, the “Cumulative Returns” plot shows positive performance but with dips into negative returns, further underscoring the agent's susceptibility to volatile conditions.
- The “Daily Returns Distribution” histogram reveals a relatively tight distribution with minimal skewness, suggesting that the model has limited outlier returns. Combined with the modest Sharpe ratio, this distribution indicates a cautious strategy with small but frequent adjustments, as opposed to a high-risk, high-reward approach.

Future Extensions and Improvements

1. Enhanced Reward Function with Risk Sensitivity:

- Future work could incorporate risk-sensitive metrics, such as adjusting the reward function to include a Sharpe or Sortino ratio-based component. This adjustment could guide the agent to prioritize risk-adjusted returns, reducing susceptibility to high drawdowns and improving the stability of returns.

2. Simulating Real-World Market Conditions:

- Introducing more realistic transaction costs, including slippage and liquidity impacts, could provide a closer approximation to live trading environments. Enhancing the simulation would mitigate the risk of overestimating returns and lead to a more reliable and deployable strategy.

3. Integration of Hybrid or Adaptive Models:

- Implementing a hybrid approach by combining PPO with other reinforcement learning algorithms, such as Soft Actor-Critic (SAC), may enhance adaptability. Hybrid models could help the agent adjust more dynamically to market conditions, blending PPO's stability with a more responsive learning process.

4. Diversifying Across Multiple Assets:

- Extending the model to a multi-asset framework, where the agent simultaneously manages a diversified portfolio, could spread risk and improve returns. This approach would allow the agent to capitalize on cross-asset opportunities, potentially enhancing the overall risk-adjusted performance.

5. Incremental Retraining with Live Data:

- Incorporating incremental retraining using recent market data would enhance the agent's adaptability in real-world applications. This rolling update approach would enable the agent to stay relevant and responsive to current market conditions, supporting better performance in evolving environments.

Conclusion

The PPO-based reinforcement learning agent demonstrated promising results in simulating automated trading within a controlled environment. The agent's conservative approach resulted in stable returns and a balanced win-loss ratio. However, the analysis and visualizations reveal areas where improvements in risk-adjusted performance, adaptability, and trading realism are needed to make the model viable for real-world deployment.

With refinements in reward functions, market simulation, and potential hybrid model implementations, reinforcement learning can evolve into a powerful tool for algorithmic trading. Future work on expanding this PPO model's capabilities will contribute to creating a more versatile and reliable trading agent, bridging the gap between academic reinforcement learning models and practical financial applications.

5 References

[References should be cited (i.e. actually be referred to) at the appropriate place in your text, they should visibly influence your document, and they should convey as much information as possible to the reader.]

Example:

[1] Shaoqing Ren, Kaiming He, Ross B. Girshick and Jian Sun, *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*, <http://arxiv.org/abs/1506.01497>, 2015

Appendices (Mandatory Section)

A. Individual Contribution:

Karan Bhutani

In this project, my primary responsibility was to bring together the diverse elements of our work into a cohesive and comprehensive report. I focused on articulating each section clearly, ensuring that the technical aspects of our project—such as data preprocessing, environment setup, model design, and results evaluation—were presented in a way that would resonate with both technical and non-technical readers.

As I compiled and structured the report, I took extra care to highlight the intricacies of our trading model and the thought process behind each development step. This involved weaving together our collective insights, visualizations, and experimental results, and explaining how each component contributed to the project's overall success. Additionally, I aimed to maintain a balanced narrative that showcased both the strengths and areas for improvement in our model, providing a well-rounded view of our findings.

Throughout the writing process, I collaborated closely with the team, gathering feedback and refining sections to ensure that the report accurately represented our shared vision and efforts. My goal was to create a document that not only serves as a technical record of our work but also reflects the depth of analysis and innovation that went into developing our autonomous stock trading agent.

Md Sadman Sakib

In this project, I implemented reinforcement learning techniques for stock trading. The approach demonstrates how to apply knowledge gathered during the learning journey.

I cooperated with Giovanni to handle both the approach development and coding sections thoroughly. The process included comprehensive data cleaning, model implementation, and detailed analysis. After testing several reinforcement learning models during the training stage, only the best-performing model was selected. The final report contains all necessary plots and results to provide complete documentation. I refined the application to showcase the project outcomes, incorporating the actions taken by the trained model. I meticulously analysed the training phases and gave enough time for the agents to learn. As project lead, I coordinated group members, managed project discussions, and oversaw the creation of flow charts.

Bianco Giovanni

In this project, I played an integral role in planning and developing the model across various stages. Collaborating closely with Sakib, we identified and implemented optimal approaches to advance the project, address issues, and enhance performance. My responsibilities included defining and testing the environment, experimenting with diverse observation types, action spaces, and reward functions to determine the most effective configurations. I also managed data preprocessing and selected key financial metrics that enabled our agent to interpret market conditions and make informed decisions. I trained and evaluated multiple models, including PPO, A2C, DDPG, SAC, and TD3, performing critical analyses to understand the

strengths and limitations of each. Using simulations, I tested the models and visualized their reward trajectories over time under various conditions and data inputs. Additionally, I designed and implemented the GUI to display essential information and simulate the agent's performance, effectively highlighting our approach's efficacy. For initial testing, I incorporated random actions as the final model was still in training.

B. Individual Contribution Split

Team Member Name	Percentage
Md Sadman Sakib	33.33
Bianco Giovanni	33.33
Karan Bhutani	33.33

All the team members have discussed and agreed on the above individual contribution to the project.

[Sakib] : [Md sadman sakib]

[Giovanni] : [Bianco Giovanni]

[Karan] : [karan bhutani]